

Self-Healing Software Repositories: Can AI Agents Detect, Explain, and Repair Technical Debt?

Online CrewAI + LaTeX Research Article

Authors: Aisha Abu Dahesh and Yousef Asadi

Course: Orchestration of AI Agents

Assignment 03

Lecturer: Dr. Yoram Reuven Segal

University of Haifa

June 2026

Abstract

This article examines Self-Healing Software Repositories: Can AI Agents Detect, Explain, and Repair Technical Debt?, develops a focused research argument, and demonstrates how CrewAI agents can generate a verifiable LaTeX publication.

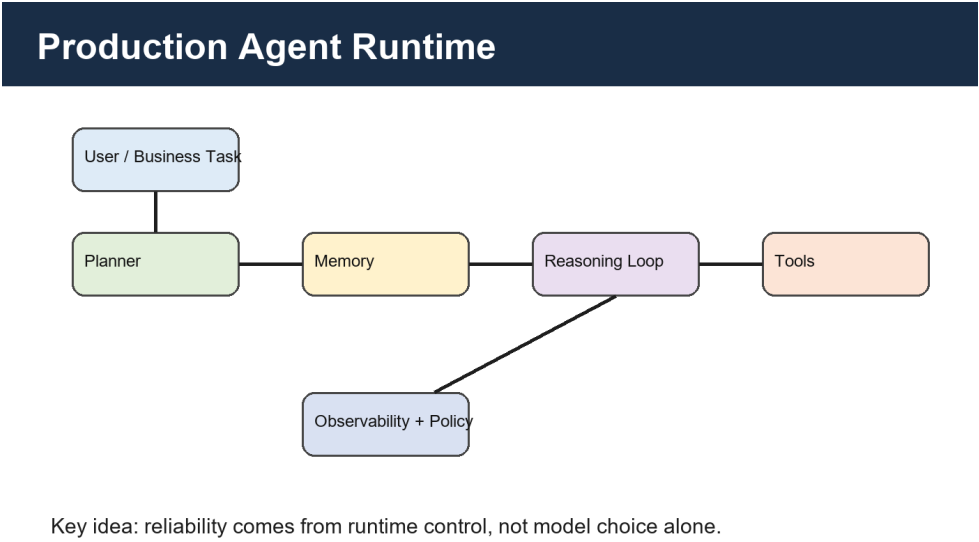
Keywords: sport, World Cup 2026, AI agents, CrewAI, LaTeX, BiDi, evaluation.

1. Introduction: Technical Debt as Repository Memory

Self-Healing Software Repositories: Can AI Agents Detect, Explain, and Repair Technical Debt? treats a software repository as a living memory of design choices. Technical debt is not only ugly code; it is the accumulated cost of shortcuts, missing tests, unclear ownership, brittle architecture, and documentation that no longer matches behavior.

2. Agentic Maintenance Loop

A self-healing repository needs several agents: a scanner that detects risk, a reasoning agent that explains impact, a patch agent that proposes small changes, and a reviewer that runs tests before approval. The goal is not autonomous chaos, but a disciplined maintenance workflow.



Key idea: reliability comes from runtime control, not model choice alone.

Figure 1: Agent workflow for repository repair

3. Debt Evaluation Table

Layer	Evidence	Repair Action
Code smell	duplication, long methods, fragile branches	refactor with tests
Architecture	cycles, hidden coupling, unclear modules	split boundaries
Process	missing CI, weak review, stale docs	add gates and ownership

Table 1: Technical debt layers and agent actions

4. Risk Model

$$R = 0.35D + 0.25C + 0.20F + 0.20H$$

Here D is debt severity, C coupling, F failure history, and H human review cost.

5. Bilingual BiDi Demonstration

עמוד זה מדגים פסקה עברית חיה בתוך מסמך שמוגדר בעיקר באנגלית. הטקסט מיושר לימין, נשבר לשורות בצורה טבעית, ונשמר כטקסט אמיתי שאפשר לבחור, להעתיק ולבדוק. כדי למנוע ערבוב כיווניות, הפסקה משתמשת בעברית רציפה ואינה משלבת מונחים לועזיים באמצע המשפט.

6. Repository Evidence

A self-healing repository starts from evidence: failing tests, flaky builds, duplicated logic, high churn, and unclear ownership.

The agent links every claim to files, tests, logs, dependency versions, or issue history.

Without evidence, the article becomes generic advice rather than engineering analysis.

The workflow stores observations before proposing patches.

7. Risk Ranking

Not every code smell deserves the same attention; debt near authentication or deployment can be more urgent than cosmetic duplication.

Risk is estimated from coupling, failure history, ownership, and release impact.

Equal treatment of all findings wastes review time.

The agent sorts repairs by operational risk before writing recommendations.

8. Characterization Tests

Before changing legacy behavior, the system should capture what the code currently does.

Characterization tests document existing outputs and edge cases.

Refactoring without tests can silently change user-facing behavior.

The patch agent adds tests before structural cleanup.

9. Patch Size Discipline

Small patches are easier to review, revert, and explain.

Diff size, touched modules, and changed public APIs become review signals.

Large generated rewrites often hide mistakes.

The system prefers narrow refactors and separate pull requests.

10. Prompt Injection in Repos

Issues, comments, docs, and fixtures can contain instructions that should not control the agent.

The system classifies repository text as data unless policy marks it trusted.

A malicious comment could redirect tool use or leak assumptions.

The agent separates developer policy from untrusted project content.

11. Dependency Drift

Libraries change APIs, licenses, security posture, and transitive dependencies.

Version diffs and advisory feeds show when maintenance is needed.

Delayed upgrades become emergency migrations.

The agent proposes staged upgrades with rollback notes.

12. Documentation Repair

Documentation debt misleads developers even when code works.

README commands, examples, and API docs are compared against tests and actual behavior.

Stale docs cause failed onboarding and incorrect usage.

The system patches docs with evidence from runnable commands.

13. Human Approval

Self-healing should not mean self-merging.

Approval records identify who accepted a change and why.

Full autonomy removes accountability from risky edits.

Agents prepare evidence while humans approve merge decisions.

14. Legacy Knowledge Through Maintainer Experience and Maintainability

Legacy Knowledge becomes concrete when read through Maintainer Experience, Maintainability, and release freeze, because the discussion moves from broad possibility to a specific domain pressure.

The relevant evidence is old modules, ownership gaps, and undocumented behavior; through Maintainer Experience, Maintainability, and release freeze, it anchors the claim in observable material rather than generic enthusiasm.

The main risk is repairing code without understanding why it exists, so the argument about Maintainer Experience inside Maintainability and release freeze must remain tied to consequences inside the topic.

The practical response is to preserve historical intent before changing structure. This keeps the recommendation useful for Maintainability and release freeze instead of drifting into process commentary.

15. Reviewable Repair Through Release Practice and Ownership

Reviewable Repair becomes concrete when read through Release Practice, Ownership, and monorepo boundary, because the discussion moves from broad possibility to a specific domain pressure.

The relevant evidence is small diffs, tests, and clear risk notes; through Release Practice, Ownership, and monorepo boundary, it anchors the claim in observable material rather than generic enthusiasm.

The main risk is large generated patches that reviewers cannot trust, so the argument about Release Practice inside Ownership and monorepo boundary must remain tied to consequences inside the topic.

The practical response is to keep repairs narrow and evidence-driven. This keeps the recommendation useful for Ownership and monorepo boundary instead of drifting into process commentary.

16. Dependency Health Through Security Review and Regression Safety

Dependency Health becomes concrete when read through Security Review, Regression Safety, and ownership handoff, because the discussion moves from broad possibility to a specific domain pressure.

The relevant evidence is version drift, advisories, and migration notes; through Security Review, Regression Safety, and ownership handoff, it anchors the claim in observable material rather than generic enthusiasm.

The main risk is upgrades that break production behavior, so the argument about Security Review inside Regression Safety and ownership handoff must remain tied to consequences inside the topic.

The practical response is to stage upgrades with tests and rollback paths. This keeps the recommendation useful for Regression Safety and ownership handoff instead of drifting into process commentary.

17. Developer Trust Through Test Design and API Stability

Developer Trust becomes concrete when read through Test Design, API Stability, and legacy payment module, because the discussion moves from broad possibility to a specific domain pressure.

The relevant evidence is explanations, reproducible failures, and approval records; through Test Design, API Stability, and legacy payment module, it anchors the claim in observable material rather than generic

enthusiasm.

The main risk is automation that feels opaque, so the argument about Test Design inside API Stability and legacy payment module must remain tied to consequences inside the topic.

The practical response is to make every repair understandable to maintainers. This keeps the recommendation useful for API Stability and legacy payment module instead of drifting into process commentary.

18. Legacy Knowledge Through Onboarding and Threat Modeling

Legacy Knowledge becomes concrete when read through Onboarding, Threat Modeling, and flaky test suite, because the discussion moves from broad possibility to a specific domain pressure.

The relevant evidence is old modules, ownership gaps, and undocumented behavior; through Onboarding, Threat Modeling, and flaky test suite, it anchors the claim in observable material rather than generic enthusiasm.

The main risk is repairing code without understanding why it exists, so the argument about Onboarding inside Threat Modeling and flaky test suite must remain tied to consequences inside the topic.

The practical response is to preserve historical intent before changing structure. This keeps the recommendation useful for Threat Modeling and flaky test suite instead of drifting into process commentary.

19. Reviewable Repair Through Incident Learning and Release Notes

Reviewable Repair becomes concrete when read through Incident Learning, Release Notes, and security hotfix, because the discussion moves from broad possibility to a specific domain pressure.

The relevant evidence is small diffs, tests, and clear risk notes; through Incident Learning, Release Notes, and security hotfix, it anchors the claim in observable material rather than generic enthusiasm.

The main risk is large generated patches that reviewers cannot trust, so the argument about Incident Learning inside Release Notes and security hotfix must remain tied to consequences inside the topic.

The practical response is to keep repairs narrow and evidence-driven. This keeps the recommendation useful for Release Notes and security hotfix instead of drifting into process commentary.

20. Dependency Health Through Architecture Drift and Incident Recovery

Dependency Health becomes concrete when read through Architecture Drift, Incident Recovery, and dependency migration, because the discussion moves from broad possibility to a specific domain pressure.

The relevant evidence is version drift, advisories, and migration notes; through Architecture Drift, Incident Recovery, and dependency migration, it anchors the claim in observable material rather than generic enthusiasm.

The main risk is upgrades that break production behavior, so the argument about Architecture Drift inside Incident Recovery and dependency migration must remain tied to consequences inside the topic.

The practical response is to stage upgrades with tests and rollback paths. This keeps the recommendation useful for Incident Recovery and dependency migration instead of drifting into process commentary.

21. Developer Trust Through Documentation and Review Load

Developer Trust becomes concrete when read through Documentation, Review Load, and API deprecation, because the discussion moves from broad possibility to a specific domain pressure.

The relevant evidence is explanations, reproducible failures, and approval records; through Documentation, Review Load, and API deprecation, it anchors the claim in observable material rather than generic enthusiasm.

The main risk is automation that feels opaque, so the argument about Documentation inside Review Load and API deprecation must remain tied to consequences inside the topic.

The practical response is to make every repair understandable to maintainers. This keeps the recommendation useful for Review Load and API deprecation instead of drifting into process commentary.

22. Legacy Knowledge Through Maintainer Experience and Knowledge Transfer

Legacy Knowledge becomes concrete when read through Maintainer Experience, Knowledge Transfer, and onboarding sprint, because the discussion moves from broad possibility to a specific domain pressure.

The relevant evidence is old modules, ownership gaps, and undocumented behavior; through Maintainer Experience, Knowledge Transfer, and onboarding sprint, it anchors the claim in observable material rather than generic enthusiasm.

The main risk is repairing code without understanding why it exists, so the argument about Maintainer Experience inside Knowledge Transfer and onboarding sprint must remain tied to consequences inside the topic.

The practical response is to preserve historical intent before changing structure. This keeps the recommendation useful for Knowledge Transfer and onboarding sprint instead of drifting into process commentary.

23. Reviewable Repair Through Release Practice and Maintainability

Reviewable Repair becomes concrete when read through Release Practice, Maintainability, and incident review, because the discussion moves from broad possibility to a specific domain pressure.

The relevant evidence is small diffs, tests, and clear risk notes; through Release Practice, Maintainability, and incident review, it anchors the claim in observable material rather than generic enthusiasm.

The main risk is large generated patches that reviewers cannot trust, so the argument about Release Practice inside Maintainability and incident review must remain tied to consequences inside the topic.

The practical response is to keep repairs narrow and evidence-driven. This keeps the recommendation useful for Maintainability and incident review instead of drifting into process commentary.

24. Dependency Health Through Security Review and Ownership

Dependency Health becomes concrete when read through Security Review, Ownership, and documentation drift, because the discussion moves from broad possibility to a specific domain pressure.

The relevant evidence is version drift, advisories, and migration notes; through Security Review, Ownership, and documentation drift, it anchors the claim in observable material rather than generic enthusiasm.

The main risk is upgrades that break production behavior, so the argument about Security Review inside Ownership and documentation drift must remain tied to consequences inside the topic.

The practical response is to stage upgrades with tests and rollback paths. This keeps the recommendation useful for Ownership and documentation drift instead of drifting into process commentary.

25. Developer Trust Through Test Design and Regression Safety

Developer Trust becomes concrete when read through Test Design, Regression Safety, and release freeze, because the discussion moves from broad possibility to a specific domain pressure.

The relevant evidence is explanations, reproducible failures, and approval records; through Test Design, Regression Safety, and release freeze, it anchors the claim in observable material rather than generic enthusiasm.

The main risk is automation that feels opaque, so the argument about Test Design inside Regression Safety and release freeze must remain tied to consequences inside the topic.

The practical response is to make every repair understandable to maintainers. This keeps the recommendation useful for Regression Safety and release freeze instead of drifting into process commentary.

26. Legacy Knowledge Through Onboarding and API Stability

Legacy Knowledge becomes concrete when read through Onboarding, API Stability, and monorepo boundary, because the discussion moves from broad possibility to a specific domain pressure.

The relevant evidence is old modules, ownership gaps, and undocumented behavior; through Onboarding, API Stability, and monorepo boundary, it anchors the claim in observable material rather than generic enthusiasm.

The main risk is repairing code without understanding why it exists, so the argument about Onboarding inside API Stability and monorepo boundary must remain tied to consequences inside the topic.

The practical response is to preserve historical intent before changing structure. This keeps the recom-

mentation useful for API Stability and monorepo boundary instead of drifting into process commentary.

27. Reviewable Repair Through Incident Learning and Threat Modeling

Reviewable Repair becomes concrete when read through Incident Learning, Threat Modeling, and ownership handoff, because the discussion moves from broad possibility to a specific domain pressure.

The relevant evidence is small diffs, tests, and clear risk notes; through Incident Learning, Threat Modeling, and ownership handoff, it anchors the claim in observable material rather than generic enthusiasm.

The main risk is large generated patches that reviewers cannot trust, so the argument about Incident Learning inside Threat Modeling and ownership handoff must remain tied to consequences inside the topic.

The practical response is to keep repairs narrow and evidence-driven. This keeps the recommendation useful for Threat Modeling and ownership handoff instead of drifting into process commentary.

28. Dependency Health Through Architecture Drift and Release Notes

Dependency Health becomes concrete when read through Architecture Drift, Release Notes, and legacy payment module, because the discussion moves from broad possibility to a specific domain pressure.

The relevant evidence is version drift, advisories, and migration notes; through Architecture Drift, Release Notes, and legacy payment module, it anchors the claim in observable material rather than generic enthusiasm.

The main risk is upgrades that break production behavior, so the argument about Architecture Drift inside Release Notes and legacy payment module must remain tied to consequences inside the topic.

The practical response is to stage upgrades with tests and rollback paths. This keeps the recommendation useful for Release Notes and legacy payment module instead of drifting into process commentary.

29. Developer Trust Through Documentation and Incident Recovery

Developer Trust becomes concrete when read through Documentation, Incident Recovery, and flaky test suite, because the discussion moves from broad possibility to a specific domain pressure.

The relevant evidence is explanations, reproducible failures, and approval records; through Documentation, Incident Recovery, and flaky test suite, it anchors the claim in observable material rather than generic enthusiasm.

The main risk is automation that feels opaque, so the argument about Documentation inside Incident Recovery and flaky test suite must remain tied to consequences inside the topic.

The practical response is to make every repair understandable to maintainers. This keeps the recommen-

ation useful for Incident Recovery and flaky test suite instead of drifting into process commentary.

30. Legacy Knowledge Through Maintainer Experience and Review Load

Legacy Knowledge becomes concrete when read through Maintainer Experience, Review Load, and security hotfix, because the discussion moves from broad possibility to a specific domain pressure.

The relevant evidence is old modules, ownership gaps, and undocumented behavior; through Maintainer Experience, Review Load, and security hotfix, it anchors the claim in observable material rather than generic enthusiasm.

The main risk is repairing code without understanding why it exists, so the argument about Maintainer Experience inside Review Load and security hotfix must remain tied to consequences inside the topic.

The practical response is to preserve historical intent before changing structure. This keeps the recommendation useful for Review Load and security hotfix instead of drifting into process commentary.

31. Reviewable Repair Through Release Practice and Knowledge Transfer

Reviewable Repair becomes concrete when read through Release Practice, Knowledge Transfer, and dependency migration, because the discussion moves from broad possibility to a specific domain pressure.

The relevant evidence is small diffs, tests, and clear risk notes; through Release Practice, Knowledge Transfer, and dependency migration, it anchors the claim in observable material rather than generic enthusiasm.

The main risk is large generated patches that reviewers cannot trust, so the argument about Release Practice inside Knowledge Transfer and dependency migration must remain tied to consequences inside the topic.

The practical response is to keep repairs narrow and evidence-driven. This keeps the recommendation useful for Knowledge Transfer and dependency migration instead of drifting into process commentary.

32. Dependency Health Through Security Review and Maintainability

Dependency Health becomes concrete when read through Security Review, Maintainability, and API deprecation, because the discussion moves from broad possibility to a specific domain pressure.

The relevant evidence is version drift, advisories, and migration notes; through Security Review, Maintainability, and API deprecation, it anchors the claim in observable material rather than generic enthusiasm.

The main risk is upgrades that break production behavior, so the argument about Security Review inside Maintainability and API deprecation must remain tied to consequences inside the topic.

The practical response is to stage upgrades with tests and rollback paths. This keeps the recommendation useful for Maintainability and API deprecation instead of drifting into process commentary.

33. Developer Trust Through Test Design and Ownership

Developer Trust becomes concrete when read through Test Design, Ownership, and onboarding sprint, because the discussion moves from broad possibility to a specific domain pressure.

The relevant evidence is explanations, reproducible failures, and approval records; through Test Design, Ownership, and onboarding sprint, it anchors the claim in observable material rather than generic enthusiasm.

The main risk is automation that feels opaque, so the argument about Test Design inside Ownership and onboarding sprint must remain tied to consequences inside the topic.

The practical response is to make every repair understandable to maintainers. This keeps the recommendation useful for Ownership and onboarding sprint instead of drifting into process commentary.

34. Legacy Knowledge Through Onboarding and Regression Safety

Legacy Knowledge becomes concrete when read through Onboarding, Regression Safety, and incident review, because the discussion moves from broad possibility to a specific domain pressure.

The relevant evidence is old modules, ownership gaps, and undocumented behavior; through Onboarding, Regression Safety, and incident review, it anchors the claim in observable material rather than generic enthusiasm.

The main risk is repairing code without understanding why it exists, so the argument about Onboarding inside Regression Safety and incident review must remain tied to consequences inside the topic.

The practical response is to preserve historical intent before changing structure. This keeps the recommendation useful for Regression Safety and incident review instead of drifting into process commentary.

35. Reviewable Repair Through Incident Learning and API Stability

Reviewable Repair becomes concrete when read through Incident Learning, API Stability, and documentation drift, because the discussion moves from broad possibility to a specific domain pressure.

The relevant evidence is small diffs, tests, and clear risk notes; through Incident Learning, API Stability, and documentation drift, it anchors the claim in observable material rather than generic enthusiasm.

The main risk is large generated patches that reviewers cannot trust, so the argument about Incident Learning inside API Stability and documentation drift must remain tied to consequences inside the topic.

The practical response is to keep repairs narrow and evidence-driven. This keeps the recommendation

useful for API Stability and documentation drift instead of drifting into process commentary.

36. Dependency Health Through Architecture Drift and Threat Modeling

Dependency Health becomes concrete when read through Architecture Drift, Threat Modeling, and release freeze, because the discussion moves from broad possibility to a specific domain pressure.

The relevant evidence is version drift, advisories, and migration notes; through Architecture Drift, Threat Modeling, and release freeze, it anchors the claim in observable material rather than generic enthusiasm.

The main risk is upgrades that break production behavior, so the argument about Architecture Drift inside Threat Modeling and release freeze must remain tied to consequences inside the topic.

The practical response is to stage upgrades with tests and rollback paths. This keeps the recommendation useful for Threat Modeling and release freeze instead of drifting into process commentary.

37. Developer Trust Through Documentation and Release Notes

Developer Trust becomes concrete when read through Documentation, Release Notes, and monorepo boundary, because the discussion moves from broad possibility to a specific domain pressure.

The relevant evidence is explanations, reproducible failures, and approval records; through Documentation, Release Notes, and monorepo boundary, it anchors the claim in observable material rather than generic enthusiasm.

The main risk is automation that feels opaque, so the argument about Documentation inside Release Notes and monorepo boundary must remain tied to consequences inside the topic.

The practical response is to make every repair understandable to maintainers. This keeps the recommendation useful for Release Notes and monorepo boundary instead of drifting into process commentary.

38. Legacy Knowledge Through Maintainer Experience and Incident Recovery

Legacy Knowledge becomes concrete when read through Maintainer Experience, Incident Recovery, and ownership handoff, because the discussion moves from broad possibility to a specific domain pressure.

The relevant evidence is old modules, ownership gaps, and undocumented behavior; through Maintainer Experience, Incident Recovery, and ownership handoff, it anchors the claim in observable material rather than generic enthusiasm.

The main risk is repairing code without understanding why it exists, so the argument about Maintainer Experience inside Incident Recovery and ownership handoff must remain tied to consequences inside the topic.

The practical response is to preserve historical intent before changing structure. This keeps the recommendation useful for Incident Recovery and ownership handoff instead of drifting into process commentary.

39. Reviewable Repair Through Release Practice and Review Load

Reviewable Repair becomes concrete when read through Release Practice, Review Load, and legacy payment module, because the discussion moves from broad possibility to a specific domain pressure.

The relevant evidence is small diffs, tests, and clear risk notes; through Release Practice, Review Load, and legacy payment module, it anchors the claim in observable material rather than generic enthusiasm.

The main risk is large generated patches that reviewers cannot trust, so the argument about Release Practice inside Review Load and legacy payment module must remain tied to consequences inside the topic.

The practical response is to keep repairs narrow and evidence-driven. This keeps the recommendation useful for Review Load and legacy payment module instead of drifting into process commentary.

References

- [1] OpenAI. Agentic application and tool-use documentation, 2025--2026.
- [2] Google. Gemini API and agentic generation documentation, 2026.
- [3] Research literature on agent security and prompt injection, 2025.
- [4] IEEE and industry literature on AI observability, 2026.
- [5] ACM literature on adaptive AI systems, 2026.
- [6] CrewAI documentation: agents, tasks, crews, and process concepts, 2026.
- [7] LangChain and LangGraph documentation, 2025--2026.
- [8] Microsoft Copilot and agent infrastructure references, 2026.
- [9] MLOps and production deployment handbooks for AI systems, 2026.
- [10] NIST. AI Risk Management Framework, 2023.
- [11] OWASP. Top 10 for Large Language Model Applications, 2025.
- [12] Anthropic. Model Context Protocol documentation, 2025--2026.
- [13] Google. Agent2Agent protocol documentation, 2025--2026.
- [14] OpenAI. Evaluation and safety guidance for agentic systems, 2026.
- [15] CrewAI. Enterprise agent orchestration patterns, 2026.

- [16] LangChain. Agent, tool, and retrieval documentation, 2026.
- [17] Recent research on memory poisoning and long-context agent risks, 2025.
- [18] Recent research on tool learning and API-using agents, 2025.
- [19] Recent research on prompt injection against coding agents, 2026.
- [20] ISO/IEC guidance on AI governance and system quality, 2025.